

Chat Service

Serviço de mensageria em tempo real — atômico, integrável e reutilizável como base de chat em qualquer aplicação do ecossistema pessoal.

Autor Emanuel Martins

Data Maio 2026

Status Planejamento

Contexto financial_control ecosystem

1 Contexto e Motivação

O financial_control já é um sistema orientado a chat: o usuário envia mensagens em linguagem natural via Telegram ou Zulip e o sistema responde com confirmações de transações. A dependência de plataformas externas (Telegram bane contas, Zulip tem limitações de hospedagem) cria um ponto frágil na arquitetura.

O objetivo deste serviço é construir um cliente nativo de chat que:

● Atômico

Funciona de forma standalone, sem depender de nenhum outro serviço do ecossistema.

● Integrável

Expõe contratos bem definidos (gRPC + eventos RabbitMQ) para ser consumido por qualquer projeto.

● Reutilizável

Base de chat para qualquer aplicação futura do ecossistema, substituindo Telegram/Zulip.

● Nível de mercado

Arquitetura de gateway com multiplexing, sequence numbers, delivery guarantees e presence tracking.

2 Análise de Protocolos

WebSocket "pelado" com JSON é suficiente tecnicamente, mas não demonstra engenharia de sistemas. O que diferencia uma plataforma de mercado é o que acontece **sobre** o transporte.

XMPP

Base: WhatsApp foi construído em cima. Federado, maduro, extensível via XEPs.

Decisão: XML verboso, ecossistema parado desde 2015. Não agrega ao portfólio.

Matrix

Base: Protocolo moderno, federado, E2E encryption nativo. Usado por governos e NASA.

Decisão: Implementar homeserver do zero = 6+ meses. Usar SDK existente elimina o aprendizado.

MTPROTO

Base: Protocolo próprio do Telegram. Binário, eficiente, criptografia customizada.

Decisão: Proprietário e complexo demais. Fora de escopo.

Discord Gateway

Base: WebSocket multiplexado + Protobuf + heartbeats + sequence numbers. Erlang/OTP backend.

Decisão: [Referência principal](#). Implementável com Phoenix Channels.

Phoenix Channels

Base: O que inspirou o modelo do Discord. Multiplexing nativo, Presence como CRDT distribuído.

Decisão: [Stack escolhida](#). Já presente no ecossistema (messenger, mailer).

O que torna uma arquitetura de chat "nível de mercado" não é o protocolo de transporte — é a **engenharia sobre ele**: delivery guarantees, sequence numbers, offline queue, presence tracking e fan-out via message broker. Essas são as peças que compõem o Discord, Slack e similares.

3 Arquitetura do Serviço

VISÃO GERAL



COMPONENTES INTERNOS

● Gateway WS

Aceita conexões, autentica via JWT (mesmo issuer do ecossistema), distribui para canais. Um processo Erlang por conexão.

● Room Manager

Gerencia canais (workspace channels, DMs, broadcast). Persiste metadados no CockroachDB.

● Presence

Phoenix Presence — CRDT distribuído sem coordenação central. Rastreia quem está online em cada canal.

● Fan-out Engine

Publica mensagens no RabbitMQ exchange. Consumidores (financial_control, outros) assinam filas próprias.

● Offline Queue

Mensagens destinadas a usuários desconectados ficam na fila. Entregue em ordem na reconexão via sequence number.

● Message Store

CockroachDB com schema por tenant (herda isolamento do `financial_control`). Histórico paginado via cursor.

4 Protocolo de Mensagens

Mensagens trafegam como Protobuf sobre WebSocket — não JSON puro. Isso garante schema versionado, serialização binária eficiente e contratos explícitos entre cliente e servidor.

ESTRUTURA DO FRAME

```
// messages.proto

message ClientFrame {
  uint64 seq = 1; // número de sequência crescente por conexão
  string channel = 2; // "workspace:uuid" | "dm:uuid" | "broadcast"
  oneof payload {
    SendMessage send = 3;
    AckMessage ack = 4;
    TypingEvent typing = 5;
    PresenceQuery presence = 6;
  }
}

message ServerFrame {
  uint64 seq = 1; // seq do servidor – cliente usa para detectar gap
  string channel = 2;
  oneof payload {
    Message message = 3;
    Ack ack = 4;
    PresenceEvent presence = 5;
    SystemEvent system = 6; // heartbeat, reconnect hint
  }
}
```

DELIVERY GUARANTEES

Cada mensagem tem um `seq` crescente por canal. O cliente rastreia o último `seq` recebido. Ao reconectar, envia `resume: { channel, last_seq }` — o servidor entrega as mensagens perdidas em ordem antes de continuar o stream normal. Isso é exatamente o modelo do Discord Gateway v10.

5 Integração com financial_control

NOVA PLATAFORMA: CHAT_NATIVE

O tenant service do financial_control já suporta múltiplas plataformas (TELEGRAM, ZULIP). A integração adiciona CHAT_NATIVE como nova plataforma — o pipeline existente não muda.

```
-- tenant service: nenhuma alteração na lógica, só novo enum value
ALTER TYPE platform_enum ADD VALUE 'CHAT_NATIVE';

-- workspace member identificado pelo user_id nativo (não platform_id externo)
INSERT INTO workspace_members (workspace_id, platform, platform_id, user_id)
VALUES ($1, 'CHAT_NATIVE', $user_id, $user_id);
```

FLUXO DE MENSAGEM FINANCEIRA

1 Usuário envia mensagem no chat nativo

Frontend Angular → WebSocket → chat service recebe o texto

2 Chat service publica evento no RabbitMQ

Exchange chat.message.inbound com workspaceId, userId, text, requestId

3 Gateway do financial_control consome o evento

Identifica plataforma CHAT_NATIVE, valida membership, enfileira em messages.inbound

4 Pipeline existente processa sem alteração

transaction-extractor → LLM → ledger (CockroachDB) → automatedmessage.outbound

5 Chat service recebe confirmação

Consume automatedmessage.outbound para CHAT_NATIVE, entrega via WebSocket ao usuário

O messenger service (Elixir) existente continua sendo responsável por Telegram e Zulip. O chat service adiciona um novo consumer no mesmo exchange — sem modificar código existente.

Elixir ~1.17

Phoenix 1.7

Phoenix Channels

Phoenix Presence

CockroachDB

RabbitMQ

Protobuf

gRPC

Docker

DECISÃO	ESCOLHA	JUSTIFICATIVA
Runtime	Elixir / Erlang OTP	Já presente no ecossistema. 1 processo leve por conexão WS — escala natural.
Real-time	Phoenix Channels	Multiplexing nativo, o mesmo modelo que inspirou o Discord Gateway.
Serialização	Protobuf	Schema versionado, binário, contratos explícitos entre cliente e servidor.
Presence	Phoenix Presence	CRDT distribuído — sem coordenação central, consistência eventual.
Banco de dados	CockroachDB	Mesmo banco do ecossistema. Schema isolado por tenant (herda padrão do ledger).
Fan-out	RabbitMQ	Já presente. Exchange topic para routing por plataforma/workspace.

ESTRUTURA DE DIRETÓRIOS

```
chat/
├─ lib/
│   └─ chat/
│       ├── application.ex          # supervision tree
│       ├── channels/
│           ├── user_socket.ex      # autenticação JWT na conexão
│           ├── room_channel.ex     # canal workspace/DM
│           └─ system_channel.ex    # heartbeat, eventos de sistema
│       ├── presence.ex             # Phoenix.Presence CRDT
│       ├── rooms/                  # Room Manager (CRUD de canais)
│       ├── messages/               # Message Store (persistência)
│       ├── delivery/
│           ├── fanout.ex            # publica no RabbitMQ
│           └─ offline_queue.ex     # fila para usuários desconectados
│       ├── integrations/
│           └─ financial_control.ex  # consumer + publisher para o ecossistema
│       └─ proto/                   # definições Protobuf geradas
├─ priv/
│   └─ migrations/                  # schema chat no CockroachDB
├─ proto/
│   └─ messages.proto               # definição dos frames
├─ config/
├─ mix.exs
└─ docker-compose.yml              # standalone: chat + CockroachDB + RabbitMQ
```

7 Contratos de Integração

Por ser um serviço atômico, o chat expõe contratos estáveis que qualquer projeto do ecossistema pode consumir.

RABBITMQ — EVENTOS PUBLICADOS

EXCHANGE	ROUTING KEY	PAYLOAD
chat.message.inbound	workspace.{id}	workspaceId, userId, text, requestId, receivedAt
chat.presence	workspace.{id}.presence	userId, status (online/offline), channelId

RABBITMQ — EVENTOS CONSUMIDOS

QUEUE	SOURCE	AÇÃO
chat.notifications.inbound	financial_control / qualquer serviço	Entrega mensagem ao usuário via WebSocket (ou offline queue)

GRPC — API DE ADMINISTRAÇÃO

```
service ChatAdmin {  
  rpc CreateRoom    (CreateRoomRequest) returns (Room);  
  rpc AddMember     (AddMemberRequest)  returns (Member);  
  rpc GetHistory    (HistoryRequest)    returns (stream Message);  
  rpc SendSystemMsg (SystemMessageRequest) returns (Ack);  
}
```


M0

Fundação

Projeto Phoenix, supervisão OTP, CockroachDB schema, Docker standalone, CI básico

M1

Conexão e Autenticação

UserSocket com JWT, Phoenix Channels básico, Protobuf encoding/decoding, heartbeat

M2

Mensagens e Persistência

Send/receive, sequence numbers, Message Store no CockroachDB, histórico paginado

M3

Delivery Guarantees

Ack explícito, offline queue, replay na reconexão, at-least-once garantido

M4

Presence e Rooms

Phoenix Presence, room management, typing indicators, online/offline status

M5

Integração financial_control

Plataforma CHAT_NATIVE no tenant service, fan-out RabbitMQ, consumer de notificações

M6

Frontend

Feature module no financial_control_admin_system (Angular), componente de chat, integração WS
